

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

Отчет по лабораторной работе 6

По дисциплине WEB-программирование

Тема работы «Работа с сокетами»

Обучающийся Петрова Наталья Глебовна

Факультет факультет инфокоммуникационных технологий

Группа K3321

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся	<u>15.04.2025</u> (дата)	<u> </u> (подпись)	<u>Петрова Н. Г.</u> (Ф.И.О.)
Руководитель	<u> </u> (дата)	<u> </u> (подпись)	<u>Марченко Е. В.</u> (Ф.И.О.)

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 Задание 1 Реализация клиентской и серверных частей	4
2 Задание 2 Решение квадратного уравнения.....	6
3 Задание 3 HTML-страница	9
4 Задание 4 Многопользовательский чат	11
ЗАКЛЮЧЕНИЕ	14

ВВЕДЕНИЕ

Цель работы

Овладеть практическими навыками и умениями реализации web-серверов с использованием web-сокетов

Задачи:

1. Реализовать клиентскую и серверную часть приложения. Клиент отправляет серверу сообщение «Hello, server». Сообщение должно отразиться на стороне сервера. Сервер в ответ отправляет клиенту сообщение «Hello, client». Сообщение должно отобразиться у клиента.
2. Реализовать клиентскую и серверную часть приложения. Клиент запрашивает у сервера выполнение математической операции, параметры, которые вводятся с клавиатуры. Сервер обрабатывает полученные данные и возвращает результат клиенту (Решение квадратного уравнения).
3. Реализовать серверную часть приложения. Клиент подключается к серверу. В ответ клиент получает http-сообщение, содержащее html-страницу, которую сервер подгружает из файла index.html.
4. Реализовать многопользовательский чат.

1 Задание 1 Реализация клиентской и серверных частей

Для выполнения лабораторной работы использовалась стандартная библиотека Python – `socket`, предназначенная для сетевого взаимодействия. Файл `server_socket.py` реализует серверную часть приложения, использующую сокет для сетевого взаимодействия. При запуске сервер начинает прослушивать указанный порт и ожидает подключения от клиента. После установления соединения он принимает сообщение от клиента, отображает его в консоли и отправляет в ответ строку «Hello, client».

```
import socket

HOST = 'localhost'
PORT = 65432

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    s.sendall(b"Hello, server")
    data = s.recv(1024)
    print(f"[CLIENT] Ответ от сервера: {data.decode()}")
```

Рисунок 1.1 — Код `client_socket.py`

Файл `client_socket.py` реализует клиентскую часть, которая устанавливает соединение с сервером по указанному адресу и порту. Клиент отправляет серверу сообщение «Hello, server», после чего ожидает ответ и выводит полученную строку от сервера в консоль.

```

import socket

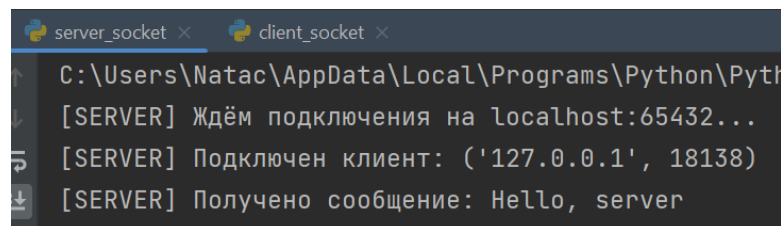
HOST = 'localhost' # 127.0.0.1
PORT = 65432

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen()
    print(f"[SERVER] Ждём подключения на {HOST}:{PORT}...")
    conn, addr = s.accept()
    with conn:
        print(f"[SERVER] Подключен клиент: {addr}")
        data = conn.recv(1024)
        print(f"[SERVER] Получено сообщение: {data.decode()}")
        conn.sendall(b"Hello, client")

```

Рисунок 1.2 — Код server_socket.py

Для запуска необходимо сначала запустить server.py в одном окне терминала, чтобы сервер начал прослушивать соединения. Затем в другом окне терминала запустить client_socket.py. В результате на стороне сервера отобразится принятое сообщение от клиента, а клиент получит и выведет ответ сервера.

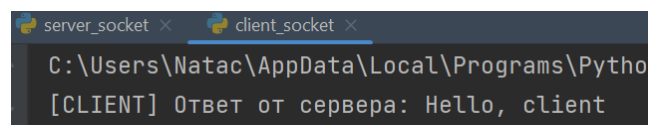


```

C:\Users\Natac\AppData\Local\Programs\Python\Python38\Scripts>python server_socket.py
[SERVER] Ждём подключения на localhost:65432...
[SERVER] Подключен клиент: ('127.0.0.1', 18138)
[SERVER] Получено сообщение: Hello, server

```

Рисунок 1.3 — Получение сообщения сервером



```

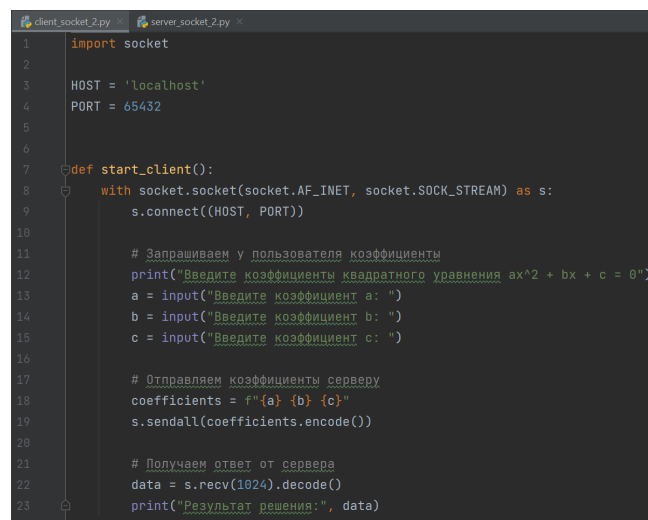
C:\Users\Natac\AppData\Local\Programs\Python\Python38\Scripts>python client_socket.py
[CLIENT] Ответ от сервера: Hello, client

```

Рисунок 1.4 — Получение сообщения клиентом

2 Задание 2 Решение квадратного уравнения

Во втором задании было реализовано клиент-серверное приложение, позволяющее выполнять вычисления корней квадратного уравнения через дискриминант. Серверная часть (`server_socket2.py`) ожидает подключения клиента, принимает три значения коэффициентов квадратного уравнения, вычисляет корни по формуле с помощью модуля `math` и отправляет результат обратно клиенту.



```
1 import socket
2
3 HOST = 'localhost'
4 PORT = 65432
5
6
7 def start_client():
8     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
9         s.connect((HOST, PORT))
10
11         # Запрашиваем у пользователя коэффициенты
12         print("Введите коэффициенты квадратного уравнения  $ax^2 + bx + c = 0$ ")
13         a = input("Введите коэффициент a: ")
14         b = input("Введите коэффициент b: ")
15         c = input("Введите коэффициент c: ")
16
17         # Отправляем коэффициенты серверу
18         coefficients = f"{a} {b} {c}"
19         s.sendall(coefficients.encode())
20
21         # Получаем ответ от сервера
22         data = s.recv(1024).decode()
23         print("Результат решения:", data)
```

Рисунок 2.1 — Код `client_socket2.py`

Клиентская часть (`client_socket2.py`) запрашивает у пользователя значения трех коэффициентов уравнения через консольный ввод, устанавливает соединение с сервером, передаёт введённые значения, получает результат вычисления и отображает его пользователю.

```

client_socket_2.py  server_socket_2.py
1  import socket
2  import math
3
4  HOST = 'localhost'
5  PORT = 65432
6
7  def solve_quadratic(a, b, c):
8      # Вычисление дискриминанта
9      discriminant = b ** 2 - 4 * a * c
10
11     if discriminant > 0:
12         # два действительных корня
13         root1 = (-b + math.sqrt(discriminant)) / (2 * a)
14         root2 = (-b - math.sqrt(discriminant)) / (2 * a)
15         return f"Два корня: x1 = {root1:.2f}, x2 = {root2:.2f}"
16     elif discriminant == 0:
17         # один корень
18         root = -b / (2 * a)
19         return f"Один корень: x = {root:.2f}"
20     else:
21         # нет действительных корней
22         return "Нет действительных корней"

```

Рисунок 2.2 — Математические расчеты корней, проверка на существование

```

client_socket_2.py  server_socket_2.py
6  with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
7      s.bind((HOST, PORT))
8      s.listen()
9      print(f"[SERVER] Ожидаем подключение на {HOST}:{PORT}...")
10     conn, addr = s.accept()
11     with conn:
12         print(f"[SERVER] Подключен клиент: {addr}")
13         data = conn.recv(1024).decode()
14         print(f"[SERVER] Получены данные: {data}")
15
16         # Разбираем коэффициенты a, b, c
17         try:
18             a_str, b_str, c_str = data.strip().split()
19             a = float(a_str)
20             b = float(b_str)
21             c = float(c_str)
22
23             # Решаем квадратное уравнение
24             response = solve_quadratic(a, b, c)
25         except Exception as e:
26             response = f"Ошибка обработки данных: {e}"
27
28         # Отправляем результат обратно клиенту
29         conn.sendall(response.encode())

```

Рисунок 2.3 — Код server_socket2.py

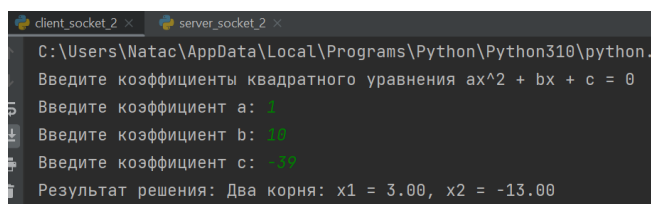
```

client_socket_2  server_socket_2
C:\Users\Natac\AppData\Local\Programs\Python\Python3
[SERVER] Ожидаем подключение на localhost:65432...
[SERVER] Подключен клиент: ('127.0.0.1', 18422)
[SERVER] Получены данные: 1 10 -39

```

Рисунок 2.4 — Запуск серверной части

Для запуска `server_socket2.py`, ожидающий подключения. Во втором — `client_socket2.py`, который отправляет значения коэффициентов и получает результат.



```
client_socket_2 x server_socket_2 x
C:\Users\Natac\AppData\Local\Programs\Python\Python310\python.
Введите коэффиценты квадратного уравнения ax^2 + bx + c = 0
Введите коэффициент a: 1
Введите коэффициент b: 10
Введите коэффициент c: -10
Результат решения: Два корня: x1 = 3.00, x2 = -13.00
```

Рисунок 2.5 — Запрос клиентских данных

3 Задание 3 HTML-страница

В данном задании необходимо реализовать простое клиент-серверное приложение. Идея в том, что сервер отправляет HTML-страничку клиенту, а клиент в свою очередь получает эту страницу, сохраняет её в файл и запускает в браузере.

```
client_socket_3.py  server_socket_3.py
1 import socket
2 HOST = 'localhost'
3 PORT = 65432
4 def start_server():
5     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
6         s.bind((HOST, PORT)) # Привязываем socket к адресу и порту
7         s.listen(1) # Сервер будет ожидать одно подключение
8         print(f"[SERVER] Ожидаем подключения на {HOST}:{PORT}...")
9
10    conn, addr = s.accept() # Принимаем подключение от клиента
11    with conn:
12        print(f"[SERVER] Подключен клиент: {addr}")
13
14        # Подгружаем HTML файл (main.html)
15        try:
16            with open('main.html', 'r', encoding='utf-8') as f:
17                html_content = f.read()
18
19            # Отправляем HTML-страницу клиенту
20            conn.sendall(html_content.encode())
21        except FileNotFoundError:
22            # Если файл не найден, отправляем ошибку 404
23            response = "404 - Страница не найдена"
24            conn.sendall(response.encode())
25
26
27 if __name__ == "__main__":
28     start_server()
```

Рисунок 3.1 — Код server_socket3.py

```
client_socket_3.py  server_socket_3.py
1 import socket
2 import webbrowser
3 import os
4
5 HOST = 'localhost'
6 PORT = 65432
7
8 def start_client():
9     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
10        s.connect((HOST, PORT)) # Подключаемся к серверу
11
12        # Получаем ответ от сервера (HTML-контент)
13        data = s.recv(1024).decode()
14        print("Результат получения страницы:")
15
16        # Сохраняем полученный HTML-контент в файл
17        with open('received_page.html', 'w', encoding='utf-8') as file:
18            file.write(data)
19
20        # Открываем файл в браузере
21        if "404" not in data:
22            webbrowser.open('file://' + os.path.realpath('received_page.html'))
23        else:
24            print(data) # В случае ошибки выводим сообщение
25
26
27 if __name__ == "__main__":
28     start_client()
```

Рисунок 3.2 — Код server_socket3.py

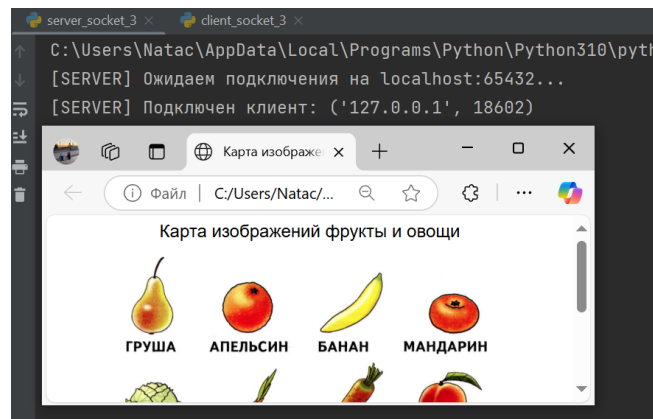


Рисунок 3.3 — Результат

4 Задание 4 Многопользовательский чат

В данном задании был реализован многопользовательский чат. Чтобы обеспечить многозадачности как на стороне сервера, так и на стороне клиента использовалась библиотека `threading`. В сервере она необходима для того, чтобы обрабатывать несколько клиентов одновременно. Каждый новый клиент, который подключается к серверу, получает свой собственный поток, что позволяет серверу параллельно обслуживать несколько пользователей без блокировки.

На стороне клиента `threading` используется для параллельного получения и отправки сообщений. Один поток отвечает за непрерывное получение сообщений от сервера и их вывод на экран, а другой поток позволяет пользователю вводить сообщения и отправлять их на сервер. Это обеспечивает удобное и плавное взаимодействие с чатом, где оба процесса происходят одновременно, не мешая друг другу.

Считывание HTML файла происходит через блок `try`, он открывает Файл `main.html` (страничка из 5 лабораторной) и считывает его содержимое через `f.read()`. После этого содержимое файла отправляется клиенту через `encode` (в бинарном виде). Клиент же со своей стороны получает данные от сервера, расшифровывает их обратно через `decode`, сохраняет их в отдельный `html` файл и открывает страничку в браузере через `webbrowser.open()`

```

1 import socket
2 import threading
3
4 clients = {}
5 def handle_client(client_socket, client_address):
6     print(f"[{client_address}] Подключен.")
7
8     username = client_socket.recv(1024).decode('utf-8')
9     clients[client_socket] = username
10    broadcast_message(f"{username} присоединился к чату.", client_socket)
11    while True:
12        try:
13            message = client_socket.recv(1024).decode('utf-8')
14            if message:
15                broadcast_message(f"{username}: {message}", client_socket)
16            else:
17                remove_client(client_socket)
18                break
19        except:
20            remove_client(client_socket)
21            break
22    def broadcast_message(message, sender_socket):
23        for client in clients:
24            if client != sender_socket:
25                try:
26                    client.send(message.encode('utf-8'))
27                except:
28                    remove_client(client)

```

Рисунок 4.1 — Код server_socket4.py

```

1 def remove_client(client_socket):
2     if client_socket in clients:
3         username = clients[client_socket]
4         del clients[client_socket]
5         print(f"[{client_socket.getpeername()}] Отключен.")
6         broadcast_message(f"{username} покинул чат.", client_socket)
7
8 def start_server():
9     server_socket = socket.socket()
10    server_socket.bind(('localhost', 1336))
11    server_socket.listen(5)
12    print("Сервер запущен и ожидает подключения...")
13
14    while True:
15        client_socket, client_address = server_socket.accept()
16        threading.Thread(target=handle_client, args=(client_socket, client_address)).start()
17
18 if __name__ == "__main__":
19     start_server()

```

Рисунок 4.2 — Код client_socket3.py

Файл client_socket4.py представляет собой клиентскую часть чата. Он устанавливает соединение с сервером и позволяет пользователю отправлять и получать сообщения. Для получения сообщений от сервера используется отдельный поток, который непрерывно слушает сообщения и выводит их на экран. В другом потоке пользователь может вводить текстовые сообщения, которые отправляются на сервер. Важно, что оба этих процесса выполняются параллельно, что позволяет пользователю одновременно читать и писать сообщения.

```

client_socket_4.py  server_socket_4.py
1 import socket
2 import threading
3 def receive_messages(client_socket):
4     while True:
5         try:
6             message = client_socket.recv(1024).decode('utf-8')
7             if message:
8                 print(message)
9             else:
10                break
11        except:
12            print("Ошибка подключения к серверу.")
13            break
14 def start_client():
15     client_socket = socket.socket()
16     client_socket.connect(('localhost', 1336))
17     username = input("Введите ваше имя: ")
18     client_socket.send(username.encode('utf-8'))
19     threading.Thread(target=receive_messages, args=(client_socket,)).start()
20     while True:
21         message = input()
22         if message.lower() == 'exit':
23             break
24         client_socket.send(message.encode('utf-8'))
25     client_socket.close()
26
27 if __name__ == "__main__":
28     start_client()

```

Рисунок 4.3 — Код client_socket4.py

```

client_socket_4 x server_socket_4 x
C:\Users\Natac\AppData\Local\Programs\Python
Сервер запущен и ожидает подключения...
[('127.0.0.1', 18509)] Подключен.
[('127.0.0.1', 18513)] Подключен.
[('127.0.0.1', 18513)] Отключен.
[('127.0.0.1', 18509)] Отключен.

```

Рисунок 4.4 — Результат работы сервера

```

client_socket_4 x server_socket_4 x
C:\Users\Natac\AppData\Local\Pr
Введите ваше имя: Наталья
Елена: Привет! Как дела?
Привет! Все хорошо, а у тебя?
C:\Windows\py.exe
Введите ваше имя: Елена
Наталья присоединился к чату.
Привет! Как дела?
Наталья: Привет! Все хорошо, а у тебя?

```

Рисунок 4.5 — Пример работоспособности чата из 2-ух пользователей

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы были получены навыки работы с WEB-сокетами, как с серверной стороны, так и с клиентской.